

```

1  import torch
2  import torch.nn as nn
3
4  class CSILSTMNet(nn.Module):
5      def __init__(self, csi_input_size, meta_input_size, window_size, num_classes,
6          hidden_size=64, num_layers=2):
7          super().__init__()
8          self.csi_lstm = nn.LSTM(csi_input_size, hidden_size, num_layers, batch_first=
9              True)
10         self.meta_lstm = nn.LSTM(meta_input_size, hidden_size, num_layers, batch_first
11             =True)
12         self.fc = nn.Linear(hidden_size * 2, num_classes)
13
14     def forward(self, csi_seq, meta_seq):
15         # csi_seq: (B, W, csi_input_size)
16         # meta_seq: (B, W, meta_input_size)
17         _, (h_csi, _) = self.csi_lstm(csi_seq) # h_csi: (num_layers, B, hidden_size)
18         _, (h_meta, _) = self.meta_lstm(meta_seq)
19         # Use last layer's hidden state
20         h_csi_last = h_csi[-1] # (B, hidden_size)
21         h_meta_last = h_meta[-1] # (B, hidden_size)
22         combined = torch.cat([h_csi_last, h_meta_last], dim=1) # (B, hidden_size*2)
23         out = self.fc(combined) # (B, num_classes)
24         return out

```